Review of Structs

Structures (aka Structs)

- A Structure is a programmer-defined data type that allows multiple variables to be grouped together.
- A structure requires a prototype and a definition like a class or function.
- To create an instance of a structure, first you prototype it, then you can declare one or more of them.
- Example

```
struct Student {  // structure name, notice the capital S
    int id;  // structure member
    std::string name;  // structure member
    float gpa;  // structure member
}; // notice the ending ;
```

Structure Declaration And Usage

- Struct names commonly begin with an uppercase letter like all ADTs.
- Variables (aka fields) in a struct are declared in a struct the same way they are normally in a program.
- All member (aka fields) in a structure are all public.
- Use the dot (.) operator to refer to the data members of struct variables:
- The member variables can be used in any manner appropriate for their data type.
- You almost never use the struct on its own, you use the form struct.field

```
Student s1;
Student s2;

s1.id = 1000;
s1.name = "bob";
s1.gpa = 2.7;

cout << s1.id << endl;
cout << s1.name << endl;
cout << s1.gpa << endl;

s2.id = s1.id;
s2.name = s1.name;
s2.gpa = s1.gpa;
```

Initializing a Struct

- Structure members can be initialized at the time a structure variable is created.
- You can initialize a structure variables' members with an initialization list.

```
Student s1 = {1000, "bob", 2.7};
```

- You can initialize a structure variables' members with a constructor.

```
struct Student {
    int id;
    std::string name;
    float gpa;

    Student(int i, std::string n, float g)
    {id = i; name = n; gpa = g;}
};

Student s2(1000, "bob", 2.7);
```

```
struct Dimensions
{ int length,
      width,
      height;
};
```

box

length	12
width	6
height	3

```
Dimensions box = {12,6,3};
```

You can initialize just some members, but you cannot skip over members

```
Dimensions box1 = {12,6};  //OK
Dimensions box2 = {12,,3}; //illegal
```

Nested Structures

A structure can have another structure as a member.

```
struct PersonInfo
{   string name,
            address,
            city;
};

struct Student
{   int         studentID;
    PersonInfo  pData;
    short       year;
    double      gpa;
};
```

```
Student s5;

s5.pData.name = "Joanne";

s5.pData.city = "Tulsa";
```

Passing Structures

- You may pass members of struct variables to functions.
  - someFunction(s1.gpa);
- You may pass entire struct variables to functions
  - someFunction(s1);
  - But you should not!
- Passing an entire structure by value can slow a program and waste space.
- Passing by reference speeds program execution, but it allows the function to modify data in the structure.
- To save space and time while protecting structure data that should not be changed, use a const reference parameter
  - void showData(const Student &s1)

Passing Structures

```
void printStruct(const Student *s){
    // notice the -> pointer notation
    cout << s->id << endl;
    cout << s->name << endl;
    cout << s->gpa << endl;
}

bool changeGpa(Student *s, float g){
    bool changed = false;
    if(g >=0 && g <=4.0){
        s->gpa = g;
        changed = true;
    }
    return changed;
}
```

```
Student s1 = {1000, "bob", 2.7};

printStruct(&s1);
changeGpa(&s1, 3.5);
printStruct(&s1);
```

```
1000
bob
2.7
1000
bob
3.5
```

Returning Structures

- A function can return a struct.
- The function must define a local structure variable for internal use and then can return it.
- If you do that, that is a pass by value operation and very slow and inefficient. You should avoid this.
- It’s better to for the caller to create an empty struct and pass it by reference to the called function. Then no return is needed except for a bool to indicate success or failure.

Returning Structures

```
Student makeStudentStructure(){
    Student s = {1000, "bob", 2.7};
    return s;
}

void fillStudentStructure(Student *s){
    *s = {1000, "bob", 2.7};
}
```

```
Student s1 = makeStudentStructure();
printStruct(&s1);

Student s2;
fillStudentStructure(&s2);
printStruct(&s2);
```